PROYECTO DE APLICACIÓN WEB PROGRESIVA

# Ruta Café

De un shell a una app
que también funciona sin conexión.

**14 fases**
con commit y verificación

**5 grupos API**
y 9 escenarios de navegador

**Fecha:** 10 de noviembre de 2026 · America/Mexico_City

**Repositorio:** Andrey-R-sys/ruta-cafe-pwa

**Versión:** 1.0.1

**Aplicación:** https://andrey-r-sys.github.io/ruta-cafe-pwa/

**Código:** https://github.com/Andrey-R-sys/ruta-cafe-pwa

**Resultado verificado.** Instalación en Chrome, catálogo, favoritos, almacenamiento local, navegación offline, sincronización con servidor local, actualizaciones y recepción push inducida por CDP.

**Alcance de Pages.** El servidor Node se entrega por separado. Reseñas compartidas y push real en Internet requieren una API HTTPS. Los casos que necesitan dispositivo físico se marcan como pendientes.

Proyecto educativo · 10 de noviembre de 2026 · America/Mexico_City

Responsable del repositorio: Andrey-R-sys. Versión del código: 1.0.1.

Aplicación: https://andrey-r-sys.github.io/ruta-cafe-pwa/

Repositorio: https://github.com/Andrey-R-sys/ruta-cafe-pwa

Estado público verificado: HTTP 200; versión observada 1.0.1; catálogo de seis lugares; hidratación, recarga offline y reseña local operativas.

# Historial de fases

| Fase | Commit inicial | Comprobaciones al cerrar la fase |
|---|---|---|
| 0 | 37389fd | 2 comprobaciones aprobadas |
| 1 | 5f28e0f | 3 comprobaciones aprobadas |
| 2 | 5df8f7b | 6 comprobaciones aprobadas |
| 3 | e973da8 | 7 comprobaciones aprobadas |
| 4 | d65e4bf | 9 comprobaciones aprobadas |
| 5 | 9a00b6e | 10 comprobaciones aprobadas |
| 6 | a20f028 | 11 comprobaciones aprobadas |
| 7 | ca23e74 | 12 comprobaciones aprobadas |
| 8 | 3e8d8a1 | 13 comprobaciones aprobadas |
| 9 | 53fe315 | 14 comprobaciones aprobadas |
| 10 | 7ce9086 | 15 comprobaciones aprobadas |
| 11 | 9d0af09 | 16 comprobaciones aprobadas |
| 12 | 57af68c | 18 comprobaciones aprobadas |
| 13 | 3bb29da | 17 comprobaciones aprobadas |

Cada fase tiene un commit identificado. Los registros HTTP y de contenido corresponden al estado de cada fase; las pruebas de comportamiento completas se realizaron en la fase 12. La publicación y sus correcciones tienen commits adicionales. El catálogo quedó incorporado al historial final tras ajustar /data/ en .gitignore.

# 1. Objetivo, alcance y arquitectura

Se construyó Ruta Café, una Aplicación Web Progresiva educativa en español que permite explorar seis cafeterías ficticias de Ciudad de México. El trabajo siguió las fases 0 a 13, con verificación y commit por fase. El catálogo y las ilustraciones son propios del proyecto; no representan comercios reales ni dependen de imágenes de terceros.

La interfaz usa HTML, CSS y módulos JavaScript sin framework. Express entrega el catálogo remoto, las fichas SSR y las API de reseñas y push. El navegador mantiene el shell en Cache Storage, las preferencias en localStorage y los registros estructurados en IndexedDB. GitHub Pages sirve una versión estática compatible con subcarpetas.

Flujo de reseñas: formulario → transacción IndexedDB (reseña y outbox) → Background Sync o reintento en primer plano → POST idempotente → almacenamiento durable en servidor → confirmación y eliminación de la cola. Si no hay servidor configurado, la reseña permanece en el dispositivo.

Flujo push: acción del usuario → permiso de notificaciones → suscripción con clave VAPID pública → registro en la API → envío administrativo autenticado → proveedor push del navegador → evento push del service worker → notificación → apertura de una URL del scope de la app.

# 2. Desarrollo por fases

## Fase 0. Entorno y repositorio

**0.1 Herramientas:** se verificaron Node v22.14.0, npm 10.9.2 y Git 2.51.0. La carpeta estaba vacía y no tenía repositorio. Se comprobó una cuenta GitHub mediante Git Credential Manager sin guardar su credencial en archivos. **0.2 Proyecto:** se inicializó Git en main y se instalaron Express, web-push, Playwright y sharp. El lockfile fija las versiones instaladas.

**0.3 Configuración:** package.json incluye start, dev, build, test, test:e2e, icons, vapid y phase. .gitignore excluye node_modules, .env, /data, dist y temporales. La barra inicial de /data/ evita excluir public/data/catalog.json. .gitattributes normaliza los archivos de texto. **0.4 Servidor mínimo:** server/index.js arranca Express en el puerto configurable; GET /api/health confirma el estado. Verificación: HTTP 200 del servidor y del shell.

## Fase 1. App shell

**1.1 HTML:** public/index.html contiene el encabezado, navegación, contenido reemplazable, pie, indicador de conexión y mensajes accesibles. Incluye enlace para saltar al contenido, idioma y viewport. **1.2 CSS:** public/css/styles.css define la paleta, tarjetas, formularios, foco visible, tema oscuro y layouts de tres, dos y una columna. El diseño evita fuentes y assets remotos. Verificación: entrega de HTML y CSS; posteriormente se verificó navegación por rutas y ausencia de desborde a 390 px.

## Fase 2. Manifiesto e íconos

**2.1 Generación:** scripts/icons.mjs convierte el SVG original en PNG de 192, 512 y 180 px; genera además la versión maskable, con el contenido importante centrado. La dependencia sharp se actualizó a una versión sin las vulnerabilidades detectadas por npm audit. **2.2 Manifiesto:** public/manifest.json declara id, name, short_name, lang, start_url, scope, display standalone, theme_color, background_color e íconos. Los valores relativos conservan el scope al publicar bajo /ruta-cafe-pwa/. Verificación: dimensiones PNG reales y lectura del manifiesto por Chrome.

## Fase 3. Service worker básico

**3.1 Registro:** js/register-sw.js registra sw.js desde una URL relativa al módulo, con updateViaCache none, detección de compatibilidad y captura de errores. **3.2 Eventos:** install prepara el shell; activate retira cachés antiguas del mismo scope; fetch permite interceptar solicitudes GET. Esta base fue ampliada en las fases 7 a 10. Verificación: sintaxis, entrega de sw.js y, en la fase 12, control efectivo de la página.

## Fase 4. CSR y SSR

**4.1 Catálogo remoto:** GET /api/catalog devuelve seis lugares; public/data/catalog.json es el respaldo transportado por HTTP en Pages. **4.2 CSR:** js/app.js interpreta las rutas #/, #/cafe/:id, #/favorites, #/near y #/settings. Renderiza tarjetas, búsqueda por nombre o colonia, categorías y estados vacíos con escape de texto.

**4.3 SSR:** GET /ficha/:id genera la ficha con nombre, descripción, imagen y dirección mediante server/ssr.js y plantillas compartidas. El contenido llega ya en el HTML y puede leerse sin JavaScript. En Pages, scripts/build.mjs genera esas fichas antes de publicar; son prerenderizadas, no SSR por petición. Verificación: respuesta de catálogo y ficha; prueba con JavaScript desactivado.

## Fase 5. Splash y Home

**5.1 Splash del sistema:** se configura mediante nombre, íconos y colores del manifiesto. El sistema operativo y navegador lo construyen; su aspecto no se controla con HTML. **5.2 Splash interno:** el shell muestra una entrada breve la primera vez en cada sesión, con tiempo máximo y sin impedir el uso cuando JavaScript está desactivado. **5.3 Home e instalación:** js/install.js captura beforeinstallprompt, ofrece un botón e indica los pasos manuales cuando esa API no existe. Detecta modo standalone y appinstalled. Verificación: instalación confirmada mediante Chrome en perfil temporal; splash Android pendiente de dispositivo físico.

## Fase 6. Almacenamiento local

**6.1 IndexedDB:** js/db.js crea los almacenes catalog, reviews, outbox y settings. Las promesas se resuelven al completar la transacción, no solo una solicitud individual. Las escrituras de reseña y cola son atómicas. **6.2 localStorage:** js/preferences.js guarda tema y favoritos; la configuración del origen API también se conserva localmente. **6.3 Respaldo remoto:** se guarda una instantánea del catálogo con fecha. Si falla la red, se consulta IndexedDB y luego el catálogo incluido. Verificación: recarga de preferencias y recuperación desde IndexedDB después de eliminar las cachés de datos y desconectar.

## Fase 7. Caché y offline

**7.1 Fallback:** offline.html explica que una página aún no está guardada y devuelve al catálogo usando el scope real del worker. **7.2 Estrategias:** cache first para el shell, network first con timeout para HTML y catálogo, stale while revalidate para imágenes. El precache incluye módulos, catálogo, íconos e ilustraciones. Las cachés de ejecución se limitan a 60 entradas por tipo; las solicitudes de escritura no se almacenan en Cache Storage. Las reseñas usan su propia persistencia en IndexedDB.

**7.3 Indicador:** los eventos online y offline actualizan el estado del shell. navigator.onLine indica conectividad del dispositivo y no garantiza que la API responda; por eso los envíos manejan también errores y timeouts. Verificación: recarga offline, ficha CSR, imágenes y fallback de una URL no visitada.

## Fase 8. Reseñas y Background Sync

**8.1 API:** GET /api/reviews?cafeId=... y POST /api/reviews validan identificador, lugar, nombre, texto y calificación. El servidor persiste JSON con escrituras serializadas y reemplazo de archivo. Una repetición de UUID devuelve el registro existente; no crea otra reseña. **8.2 Guardar primero:** el formulario escribe reseña y outbox en una transacción, muestra Pendiente y después intenta sincronizar. Solo se elimina la cola tras recibir una confirmación del mismo identificador.

**8.3 Evento sync:** el worker escucha sync-reviews, procesa la cola y avisa a las ventanas. Un candado del navegador evita ejecuciones simultáneas cuando está disponible. Los errores transitorios conservan la cola; rechazos permanentes se marcan para revisión. Hay reintentos al recuperar red, regresar al primer plano o pulsar Sincronizar ahora. Verificación: registro offline persistente, reconexión sin duplicados y evento sync real de Chromium disparado por CDP.

## Fase 9. Actualización y ciclo de vida

**9.1 Activación:** un nuevo worker queda en espera. El mensaje SKIP_WAITING se envía cuando el usuario acepta la actualización; activate ejecuta clients.claim después de limpiar sus cachés antiguas. No se borra IndexedDB ni localStorage. **9.2 Aviso:** se muestran Actualizar ahora y Después. controllerchange recarga una sola vez si la página ya estaba controlada; la primera instalación no causa una recarga innecesaria. El servidor entrega sw.js con no-cache. Verificación: nueva versión en espera, aceptación, cambio de controlador y favorito conservado.

## Fase 10. Notificaciones locales y push

**10.1 Flujo:** js/notifications.js implementa permiso bajo acción explícita, aviso local, alta y baja push. **10.2 VAPID:** npm run vapid genera claves y una clave administrativa en .env sin sobrescribir claves existentes. La privada nunca pasa al cliente ni a GitHub. **10.3 Servidor:** GET /api/push/key entrega solo la pública; POST y DELETE /api/push/subscribe administran suscripciones; POST /api/push/send exige Bearer PUSH_ADMIN_KEY. Se deduplican endpoints y se eliminan los expirados tras 404 o 410.

**10.4 Cliente:** maneja falta de soporte, permiso denegado, API sin configurar y errores de suscripción. **10.5 Worker:** push interpreta el mensaje y muestra una notificación; notificationclick enfoca una ventana o abre la app. Los destinos se restringen al origen y scope propios. Verificación: aviso local y recepción en worker mediante CDP; API y eliminación de expiradas con transportador sustituido. La entrega real de un proveedor externo requiere la prueba manual CP21 y servidor accesible.

## Fase 11. Dispositivo e hidratación

**11.1 Módulo:** js/device.js encapsula geolocalización, distancia Haversine, Web Share con alternativa de portapapeles y vibración breve opcional. **11.2 Cerca de mí:** solicita ubicación solo al pulsar el botón, calcula y ordena distancias en el navegador. No persiste ni envía coordenadas; explica permisos denegados y errores. **11.3 Hidratación:** la ficha SSR incluye un bloque JSON seguro; el cliente conserva el HTML inicial y añade favoritos, reseñas y compartir. Verificación: coordenadas simuladas, denegación de ubicación y favorito operativo sobre HTML SSR.

## Fase 12. Pruebas de software

**12.1 Herramientas:** Node Test Runner prueba la API; Playwright opera Chrome y usa CDP para manifiesto, sync, actualizaciones y recepción push. Una prueba adicional instala la PWA en un perfil temporal y la desinstala al finalizar. **12.2 Reporte:** se registran precondiciones, procedimiento, resultado esperado, resultado observado, estado y evidencia. La tabla completa se incluye en la sección 4 y en docs/PRUEBAS.md. Resultado: cinco grupos API y nueve escenarios de navegador aprobados, además de instalación y prueba estática en subcarpeta.

## Fase 13. Publicación

**13.1 GitHub:** se creó un repositorio público en la cuenta verificada y se subió el historial de fases. **13.2 Pages:** se habilitó GitHub Actions como fuente; el workflow instala dependencias, ejecuta pruebas API, construye dist, sube el artefacto y publica. Se verificó además una copia exportada del commit para detectar archivos que estuvieran solo en el entorno local.

**13.3 Alcance del hosting:** Pages ofrece shell, instalación, catálogo, preferencias, datos locales y funcionamiento offline. Las reseñas compartidas y el envío push requieren el servidor Node en otro hosting HTTPS. Las fichas de Pages se generan en build y después se hidratan. El servidor está entregado y funciona localmente; no se desplegó un backend externo ni se afirma que Pages ejecute Node.

**13.4 Actualización:** scripts/release.mjs actualiza la versión del paquete, lockfile y worker. Se publica una actualización 1.0.1 tras la versión inicial 1.0.0 para ejercitar el flujo de entrega. El mecanismo de aceptación en el cliente se verificó en la prueba CP13. Los resultados públicos y la versión servida se guardan en docs/evidencias/pages-publico.json.

# 3. Reproducción, operación y límites

## Arranque local

```
npm ci
npm run vapid
npm start
```

Abrir http://localhost:3000. Para desarrollo con reinicio usar npm run dev. Antes de desconectar, esperar la preparación del service worker y visitar el catálogo. No abrir index.html mediante file://: el worker y varias API necesitan un origen HTTP local o HTTPS.

## Uso de reseñas y push

Abrir una ficha, introducir nombre, calificación y al menos diez caracteres de reseña. Sin conexión se conserva como pendiente. Con el servidor local activo se envía al recuperar red. En Pages, configurar en Ajustes el origen de una API HTTPS, sin /api al final; el servidor debe permitir https://andrey-r-sys.github.io en ALLOWED_ORIGINS.

Para push, activar la suscripción desde Ajustes y conceder permiso. En el entorno del servidor ejecutar:

```
node --env-file=.env scripts/send-push.mjs
```

Un resultado sent: 0 no demuestra recepción: significa que no hubo suscripciones a las que enviar. Se debe comprobar una suscripción real y la notificación en el dispositivo. Mantener privadas las claves VAPID y administrativa. Al cambiar las claves VAPID hay que renovar las suscripciones.

## Publicar una nueva versión

```
node scripts/release.mjs 1.0.2
npm test
npm run build
git add .
git commit -m "Actualizar PWA a 1.0.2"
git push origin main
```

Consultar el resultado del workflow en Actions y verificar la nueva versión de sw.js. La app ofrece la actualización instalada; al aceptarla se conserva la información local.

## Límites prácticos

La persistencia del servidor usa un archivo JSON y está diseñada para una sola instancia educativa. Un despliegue con varias instancias requiere una base compartida. En hosting de disco efímero debe configurarse un volumen persistente para DATA_DIR. No hay cuentas, moderación ni administración visual de reseñas.

Background Sync no tiene soporte universal ni ejecución inmediata garantizada. El almacenamiento del navegador puede ser borrado por el usuario o el sistema. Una primera visita completamente offline no puede descargar la app. Las notificaciones requieren permiso, un navegador compatible y políticas del sistema que permitan entregarlas. Las coordenadas y distancias de los lugares son ejemplos, no recomendaciones de establecimientos reales.

# 4. Tabla de casos de prueba y evidencias

## Casos ejecutados y pendientes

| ID | Caso y precondición | Procedimiento | Resultado esperado | Resultado obtenido | Estado / evidencia |
|---|---|---|---|---|---|
| CP01 | Servidor y catálogo | Iniciar servidor; GET /api/catalog | 200; seis lugares | Seis registros válidos | Aprobado; api.tap |
| CP02 | App shell y CSR | Buscar Lumbre; filtrar Panadería; volver a Todos | 1, 2 y 6 resultados respectivamente | Conteos correctos, sin errores JS | Aprobado; home.png |
| CP03 | Manifiesto e íconos | Consultar manifiesto en CDP y dimensiones PNG | Sin errores del manifiesto; PNG 192 y 512; maskable | Formato y dimensiones correctos | Aprobado; suite API y navegador |
| CP04 | Instalación | Perfil temporal persistente; PWA.install de Chrome | Chrome registra la app instalada | Instalación confirmada; desinstalación al terminar | Aprobado; instalacion.json |
| CP05 | Favoritos y tema | Guardar favorito; tema oscuro; recargar | Se conservan preferencias | Persisten ambos valores | Aprobado; Playwright |
| CP06 | Navegación offline | Esperar SW; desconectar contexto; recargar ficha | Shell y ficha CSR disponibles | Ficha y formulario operativos | Aprobado; offline.png |
| CP07 | Respaldo IndexedDB | Borrar solo caché de datos; desconectar; recargar | Recuperar seis lugares desde IndexedDB | Seis tarjetas renderizadas | Aprobado; Playwright |
| CP08 | Fallback offline | Sin red; abrir URL no visitada | Página offline; enlace al catálogo | Página y salida al catálogo operativas | Aprobado; Playwright |
| CP09 | Reseña local primero | Sin red; escribir y recargar | Una reseña pendiente sobrevive | Cola y reseña conservadas | Aprobado; offline.png |
| CP10 | Recuperar red | Reconectar después de CP09 | Cola vacía; una reseña en servidor | Un registro, sin duplicados | Aprobado; Playwright |
| CP11 | Background Sync | Insertar reseña pendiente; disparar sync vía CDP | El SW envía y confirma la reseña | Cola vacía | Aprobado; evento real de Chromium inducido por CDP |
| CP12 | Idempotencia y persistencia | Cuatro POST simultáneos con el mismo UUID | Una creación; tres confirmaciones; un registro durable | 201, 200, 200, 200; un registro al reabrir el almacenamiento | Aprobado; api.tap |
| CP13 | Actualización | Cambiar versión servida; aceptar aviso | Nuevo worker controla; favorito conservado; caché antigua eliminada | Comportamiento confirmado | Aprobado; Playwright |
| CP14 | Notificación local | Permiso concedido en perfil de prueba; pulsar aviso | Notificación visible para el registro | Notificación creada | Aprobado; Playwright |

| ID | Caso y precondición | Procedimiento | Resultado esperado | Resultado obtenido | Estado / evidencia |
|---|---|---|---|---|---|
| CP15 | Recepción push | Entregar mensaje al SW mediante CDP | Evento push muestra aviso; bloquea URL externa | Aviso creado; destino limitado a la app | Aprobado en recepción simulada; no prueba al proveedor push |
| CP16 | Servidor push | Alta duplicada, envío con/sin clave, error 410 | Deduplicar; 401 sin clave; borrar suscripción expirada | Resultados correctos | Aprobado con transportador sustituido; api.tap |
| CP17 | SSR e hidratación | Abrir ficha sin JS y después con JS | Contenido sin JS; formulario y favorito al hidratar | Ficha legible y luego interactiva | Aprobado; ssr.png |
| CP18 | Geolocalización | Coordenadas simuladas de Lumbre; luego denegar | Orden por distancia; error comprensible | 0.0 km primero; mensaje de permiso denegado | Aprobado; Playwright |
| CP19 | Diseño móvil | Viewport 390 x 844 | Sin desplazamiento horizontal | Sin desborde | Aprobado; mobile.png |
| CP20 | Validación y CORS | JSON roto, datos inválidos y origen ajeno | Rechazar con 400 o 403 | Códigos esperados | Aprobado; api.tap |
| CP21 | Push real completo | HTTPS; servidor y VAPID; dispositivo compatible | Suscribir navegador; enviar push administrativo; recibir con app cerrada | Requiere servidor accesible y permiso real | Pendiente de validación en dispositivo; procedimiento abajo |
| CP22 | Splash del sistema | Instalar y abrir en Android compatible | Splash construido con nombre, ícono y colores del manifiesto | Configuración implementada; no se emuló Android | Pendiente de validación en dispositivo |

## Validaciones manuales pendientes

**CP21:** iniciar el servidor con `npm run vapid` y `npm start`, o alojarlo en HTTPS con disco persistente y las variables de `.env.example`. En la app, abrir Ajustes, configurar su origen HTTPS si corresponde y pulsar Activar push. Conceder permiso. En el servidor, enviar el mensaje mediante `node --env-file=.env scripts/send-push.mjs`. Cerrar las ventanas de la PWA y confirmar recepción del aviso; al pulsarlo debe abrir Ruta Café. Mantener el navegador habilitado para recibir notificaciones. En iOS compatible, usar la PWA agregada a inicio.

**CP22:** instalar desde Chrome Android, cerrar y abrir desde el ícono. Verificar nombre, ícono centrado y fondo verde. El navegador y el sistema operativo deciden cómo se presenta el splash del sistema; no se dibuja con HTML.

**Compatibilidad adicional:** repetir CP06, CP09, CP10, CP14 y CP21 en un dispositivo físico. Background Sync no está en todos los navegadores y su ejecución no es inmediata ni garantizada por el sistema. Si no está disponible, la app reintenta al recuperar red, volver a primer plano o pulsar Sincronizar ahora.

Las pruebas automatizadas certifican el comportamiento del código en Chrome. La entrega push desde Internet, el splash Android, vibración física y Web Share nativo requieren la validación de dispositivo descrita. No se presenta una simulación como entrega real de un proveedor.

# 5. Archivos, trazabilidad y fuentes

## Entregables

• Código e historial completo en el repositorio GitHub indicado en la portada.

• Aplicación publicada en GitHub Pages, con shell y catálogo preparados para uso offline.

• Reporte PDF en output/pdf/reporte-ruta-cafe.pdf y fuente Markdown en docs/REPORTE.md.

• Tabla editable en docs/PRUEBAS.md; verificaciones por fase en docs/fases.json.

• Capturas home.png, mobile.png, offline.png y ssr.png en docs/evidencias.

• Resultados API (api.tap), navegador (playwright-results.json), instalación (instalacion.json) y publicación (pages-publico.json).

• .env.example y scripts para generar claves, probar el envío push, construir, verificar y publicar actualizaciones.

## Referencias técnicas

• MDN, Background Synchronization API: https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API

• MDN, ServiceWorker y contexto seguro: https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorker

• MDN, beforeinstallprompt: https://developer.mozilla.org/en-US/docs/Web/API/Window/beforeinstallprompt_event

• MDN, skipWaiting: https://developer.mozilla.org/en-US/docs/Web/API/ServiceWorkerGlobalScope/skipWaiting

• GitHub Docs, naturaleza estática de Pages: https://docs.github.com/en/pages/getting-started-with-github-pages/what-is-github-pages

• GitHub, workflow oficial de referencia: https://github.com/actions/starter-workflows/blob/main/pages/static.yml

• web-push, biblioteca y VAPID: https://github.com/web-push-libs/web-push

• Chrome DevTools Protocol, instalación PWA: https://chromedevtools.github.io/devtools-protocol/tot/PWA/

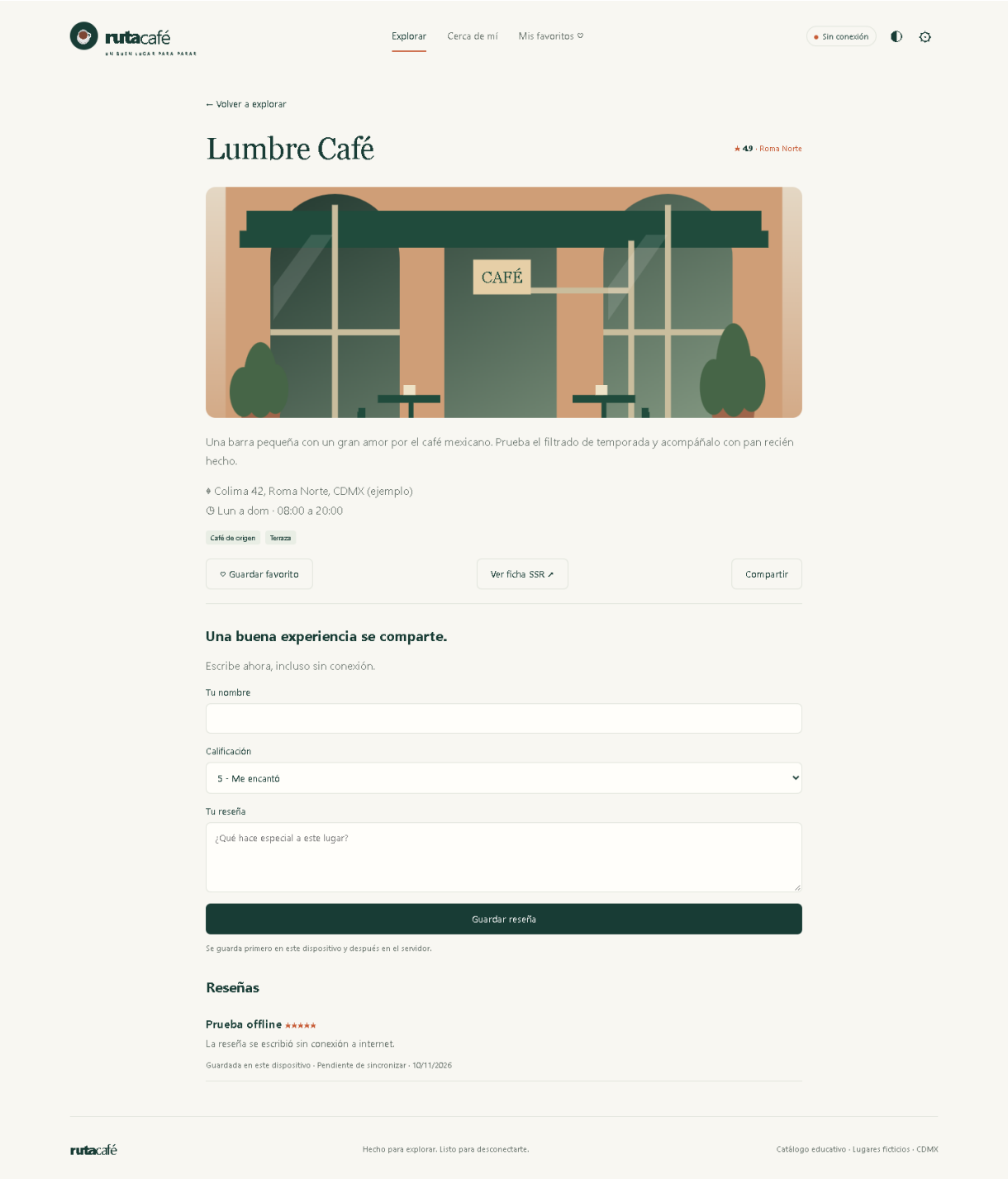
Las fuentes respaldan las decisiones de compatibilidad y publicación. Los resultados de prueba de este reporte provienen de los archivos de evidencia del proyecto, no de las referencias externas.

# Anexo visual · Evidencias de ejecución

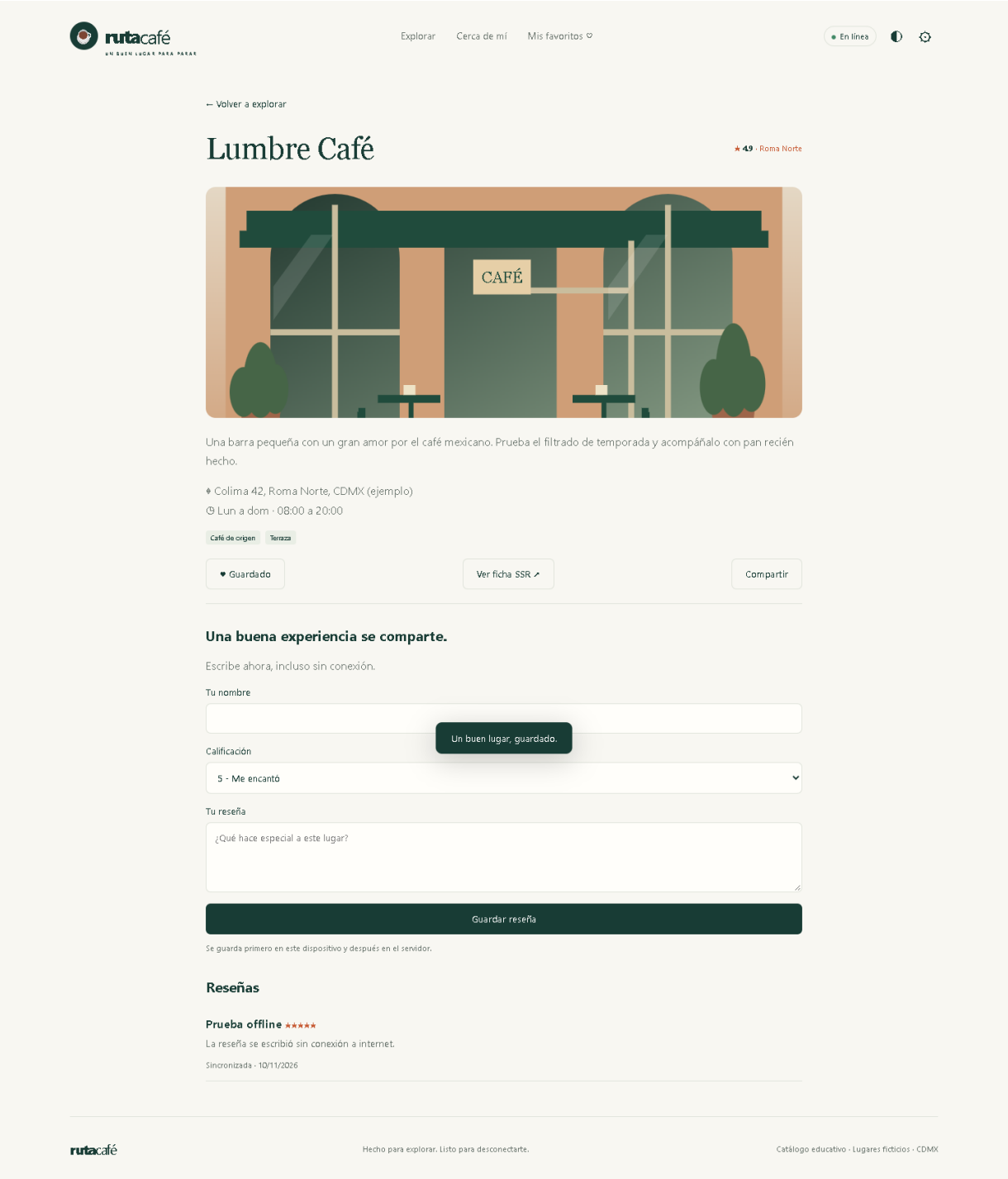
Las capturas completas están disponibles en docs/evidencias; estas vistas documentan la interfaz y el estado offline.




Home: catálogo de seis lugares, filtros e instalación.




Ficha recargada sin conexión y reseña conservada en el dispositivo.




Ficha con HTML de servidor e interacciones hidratadas.